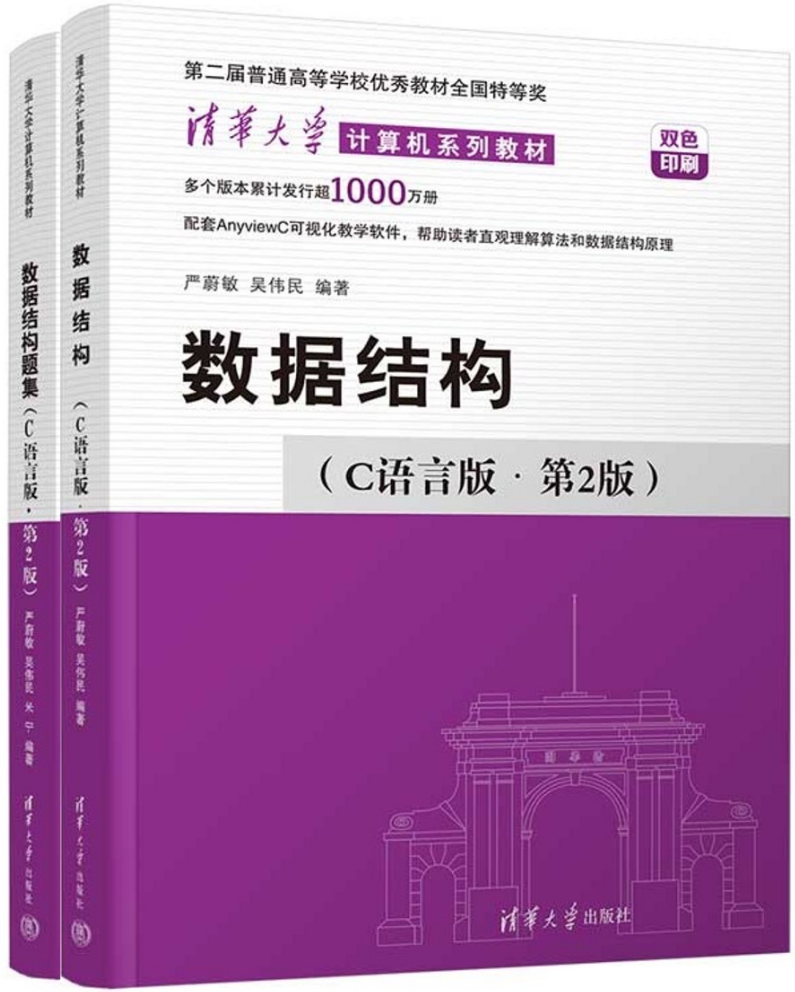


清華大學出版社
TSINGHUA UNIVERSITY PRESS

第3章

栈和队列

第3章　栈和队列

清華大學出版社
TSINGHUA UNIVERSITY PRESS

栈和**队列**是两种重要的**线性结构**。

从结构特点看，栈和队列也属**线性表**。

其特殊性在于栈和队列的基本操作是线性表操作的子集，它们是**操作受限**的线性表，因而也称为**限定性**的数据结构。

但从数据类型看，它们是和线性表不同的两类重要的**抽象数据类型**。

由于广泛应用在各种软件系统中，因此在面向对象的程序设计中，它们是**多型数据类型**。

本章除了讨论栈和队列的**定义**、**表示方法**和**实现**外，还将给出一些**应用**的例子。

3.1 栈

3.1.1 抽象数据类型栈的定义

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。

表尾端有其特殊含义，称为**栈顶**（**top**）；

表头端相应称为**栈底**（**bottom**）。

不含元素的空表称为**空栈**。

对于栈**S** = ($a_1, a_2, \ldots, a_n$)，称a_1为**栈底元素**，a_n为**栈顶元素**。栈中元素按$a_1, a_2, \ldots, a_n$的次序进栈，退栈的第一个元素应为栈顶元素。

栈的修改按后进先出的原则进行（如图3.1(a)所示）。栈又称为**后进先出**（**last in first out**）的**线性表**（简称**LIFO**结构），

可用图3.1(b)的铁路调度站形象表示。

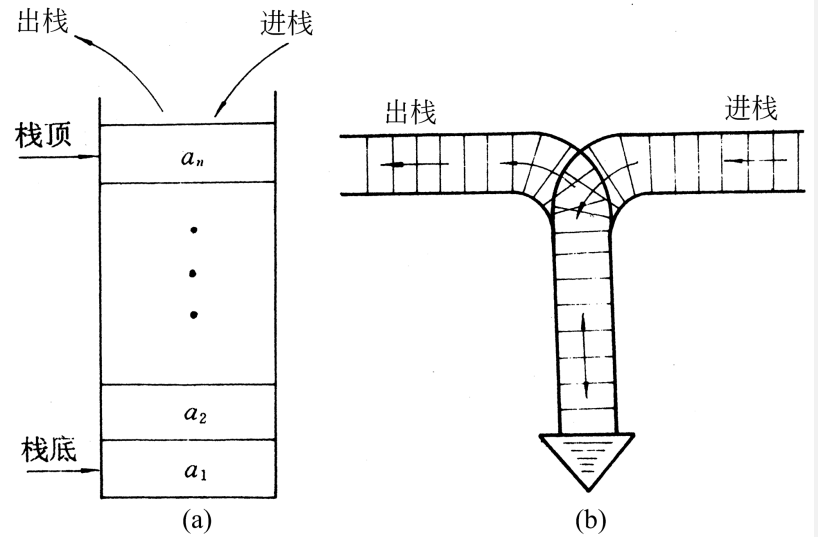


图3.1 栈

(a) 栈的示意图； (b) 用铁路调度站表示栈

栈的抽象数据类型：

ADT Stack {

数据对象：D={$a_i | a_i \in ElemSet, i=1,2,\ldots,n,\ n \geq 0$**}**

数据关系：R1={$<a_{i-1}, a_i> | a_{i-1}, a_i \in D,\ i=2,\ldots,n$ **}**

约定a_n端为栈顶，a_1端为栈底。

基本操作：

InitStack(size);

操作结果：构造并返回一个空栈，容量为size。

FreeStack(S);

初始条件：栈S已存在。

操作结果：回收栈S占用的存储空间，栈S不再可用，返回NULL。

ClearStack(S);

初始条件：栈S已存在。

操作结果：将S清为空栈。

StackEmpty(S);

初始条件：栈S已存在。

操作结果：若栈S为空栈，则返回TRUE，否则FALSE。

StackLen(S);

初始条件：栈S已存在。

操作结果：返回S的元素个数，即栈的长度。

称**插入元素**的操作为**入栈**，**删除栈顶元素**的操作为**出栈**。

栈的基本操作除了在栈顶进行插入或删除外，常用的还有**栈的初始化**、**判空**及**取栈顶元素**等。

GetTop(S);

初始条件：栈S已存在且非空。

操作结果：返回S的栈顶元素。

Push(S, e);

初始条件：栈S已存在。

操作结果：插入元素e为新的栈顶元素。每当容量不足时，按约定方式扩容。

Pop(S);

初始条件：栈S已存在且非空。

操作结果：删除S的栈顶元素，并返回其值。

StackTrav(S, visit());

初始条件：栈S已存在且非空。

操作结果：从栈底到栈顶依次对S每个数据元素调用函数visit(),一旦失败，则操作终止。

}ADT Stack

3.1.2 栈的表示和实现

和**线性表**类似，**栈**也有两种存储表示方法——**顺序栈**和**链栈**。

- **顺序栈**，即栈的顺序存储结构。
- 利用一组地址连续的存储单元依次存放**自栈底到栈顶**的数据元素。
- 附设指针**top**指示栈顶元素在顺序栈中的位置。通常的习惯做法是以**top=0**表示**空栈**。
- 栈所需最大空间很难估计，初始化设空栈时不应限定栈的最大容量。
- 合理做法是类似于顺序表：先为栈分配一个**基本容量**，每当栈的空间不够时**定增**或**倍增扩容**。

以下类型说明定义了顺序栈。

```
typedef struct {
  ElemType    *base;    // 基址，也称为栈底指针，初始化时指向分配的元素存储空间
  ElemType    *top;     // 栈顶指针
  int          size;    // 元素存储空间容量
} *SqStack               // 顺序栈指针类型
```

栈的初始化

栈的初始化操作：

构造并返回一个空栈，按申请容量分配元素存储空间。

- base称为栈底指针，始终指向栈底的位置。
- top称为栈顶指针，其初值指向栈底。
- top==base是栈空的状态。
- 每当插入新的栈顶元素时，top增1；
- 删除栈顶元素时，指针top减1。
- 非空栈的栈顶指针始终指向栈顶元素的 下一个位置。

图3.2展示了顺序栈的栈顶指针和栈顶元素之间的关系。

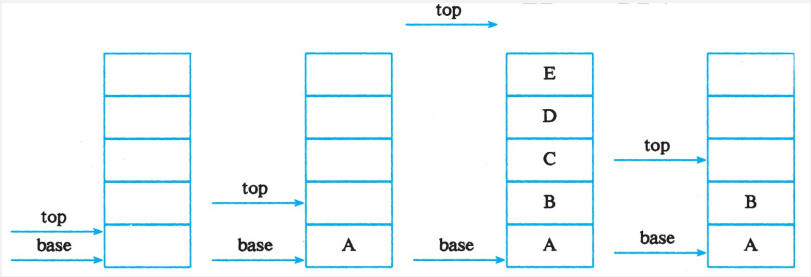
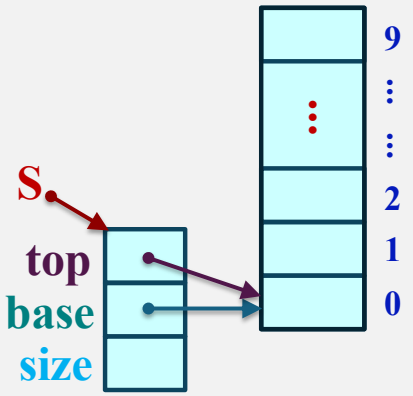


图3.2　栈顶指针和栈顶元素之间的关系

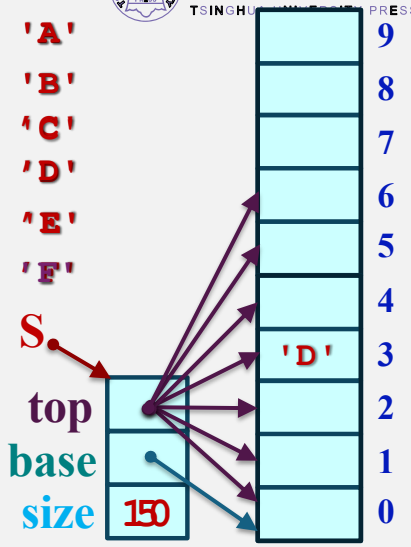
```
SqStack InitStack(int size) {
  // 构造并返回一个空顺序栈，容量为size。
  SqStack S;
  if (!(S = (SqStack)malloc(sizeof(*S))))
    exit(OVERFLOW);    // 分配存储空间
  if (!(S->base = (ElemType*)calloc(size,
    sizeof(ElemType)))) exit(OVERFLOW);
  S->top = S->base;    S->size = size;
  return S;            // 返回新建的顺序栈
}
```



入栈、出栈和取栈顶操作的实现

```
Status Push(SqStack S, ElemType e) { // 入栈，元素e置为新的栈顶元素
  if (!S) return ERROR;                // 若栈s不存在，则报错
  if (S->top - S->base >= S->size) { // 若栈已满，则申请扩容
    if (!(S->base = (ElemType*)realloc(S->base,      // 若扩容失败
          (S->size*2*sizeof(ElemType)))) return ERROR; // 则报错
    S->top = S->base + S->size;    // 更新栈顶指针（因栈底指针已改变）
    S->size *= 2;    // 更新栈的容量
  }
  *S->top++ = e;     // 将元素e置到栈顶，top增1
  return OK;
}
```

依次入栈：'A' 'B' 'C' 'D' 'E' 'F'




```
ElemType Pop(SqStack S) { // 若栈不空，则删除S栈顶元素，返回其值；否则返回nullE
  if (!S || S->top == S->base) return nullE;  // 栈不存在或空，则返回"空"元素
  return *--S->top;                           // 返回退栈元素值，满足不同需求
}
```

Pop(S) =

Pop(S) =

```
ElemType GetTop(SqStack S) { // 若栈S不空，则返回栈顶元素，否则返回空元素nullE
  if (!S || S.top==S.base) return nullE;  // 栈不存在或空，则返回"空"元素
  return *(S.top-1);                      // 返回栈顶元素
}
```

GetTop(S) =

3.2.1 数制转换

十进制数*N*和其他*d*进制数的转换的一个简单算法基于下列原理：

$$N = (N / d) * d + N \% d$$

（其中：/ 为整除运算，*为乘法运算，%为求余运

例如：$(1348)_{10} = (2504)_8$,算其运算过程如下：

N	N / 8	N % 8
1348	168	4
168	21	0
21	2	5
2	0	2

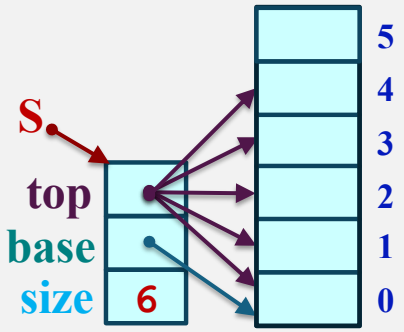
编程：对输入的任意非负十进制整数，打印输出等值的八进制数。

上述计算从低位到高位顺序产生八进制数的各个数位，打印输出通常由高位到低位进行，恰好相反。

若将计算过程中得到的八进制数的各位顺序进栈，则按出栈序列打印输出的即为对应的八进制数。

1348

```
void conversion(unsighed int N) { //算法3.1
  // 对于非负十进制整数N，打印输出等值的八进制数
  SqStack S = InitStack(6); // 建容量6的空栈
  while (N) {          // 循环中，N不断变小，直至为0
    Push(S, N%8);      // 8除N，余数入栈
    N = N/8;           // 8除N，商替代原N值
  }
  while (!StackEmpty(S))  // 循环到栈空为止
    printf("%d", Pop(S)); // 输出退栈值
  printf("\n");      // 换行，结束
}
```



依次出栈输出各位：

3.3 栈与递归的实现

清華大學出版社 TSINGHUA UNIVERSITY PRESS

栈最重要的应用之一是在程序设计语言中辅助实现函数调用，特别是**递归**调用。一个直接调用自己或通过一系列调用语句间接地调用自己的函数，称做**递归函数**。

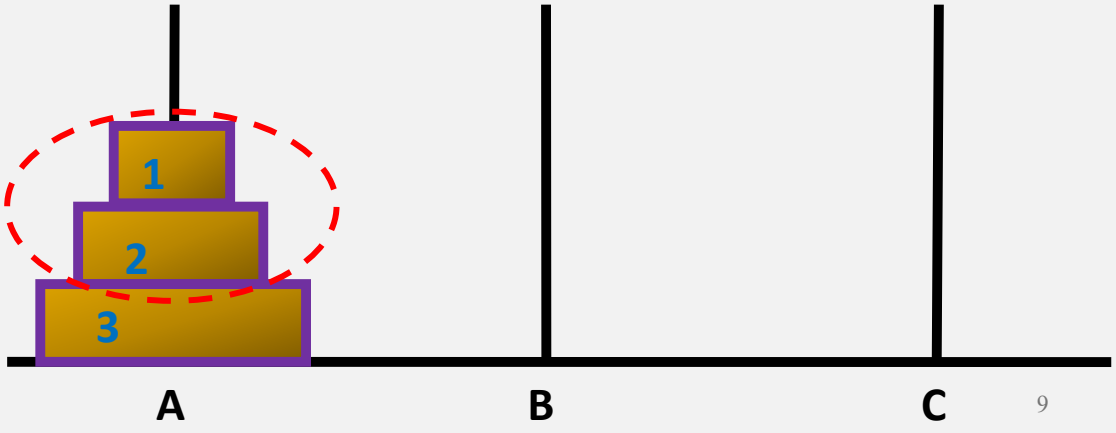
例3-2 （*n*阶Hanoi塔问题）先陈述3阶的情形：

假设有3个分别命名为X、Y和Z的塔柱，在塔柱X上插有*n*个直径大小各不相同、依小到大编号为1,2,….*n*的带孔圆盘（如图3.5所示）。要求将X柱上的*n*个圆盘移至塔柱Z上并仍按同样顺序叠放，圆盘移动时必须遵循下列规则：

（1）每次只能移动塔柱顶上的一个圆盘；

（2）圆盘可以插在X、Y和Z中的任一塔柱上；

（3）任何时刻都不能将一个较大的圆盘压在较小的的圆盘之上。



hanoi(3, A, B, C)执行过程

```
void hanoi(int n, ←实参: 3
           char x, ←--A
           char y, ←--B
           char z  ←--C
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参: 2 A C B
4    hanoi(n-1,x,z,y);
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

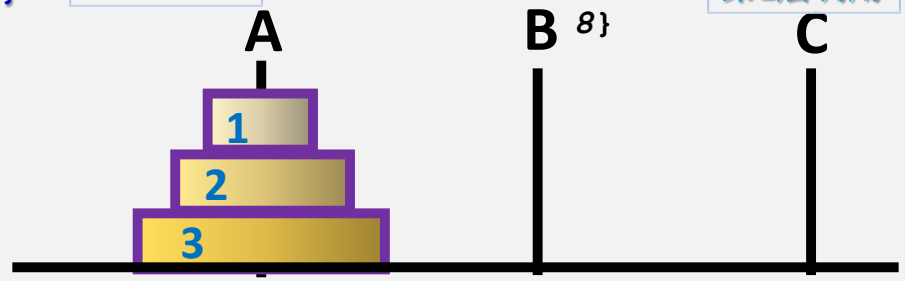
第0层调用

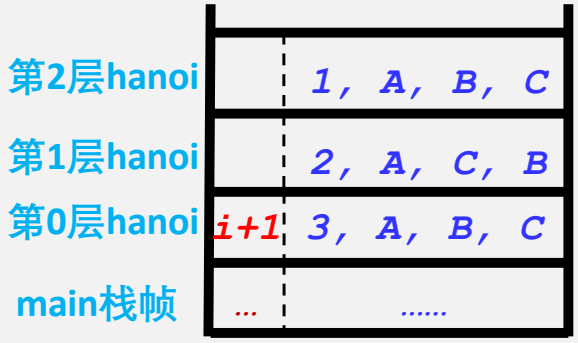
```
void hanoi(int n, ←实参: 2
           char x, ←--A
           char y, ←--C
           char z  ←--B
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参: 1 A B C
4    hanoi(n-1,x,z,y);
     //实参:
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第1层调用

```
void hanoi(int n, ←实参:
           char x, ←--
           char y, ←--
           char z  ←--
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参:
4    hanoi(n-1,x,z,y);
     //实参:
5    move(  x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第2层调用





hanoi(3, A, B, C)执行过程

```
void hanoi(int n, ←实参 3
           char x, ←--A
           char y, ←--B
           char z  ←--C
          ){
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参: 2 A C B
4    hanoi(n-1,x,z,y);
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第0层调用

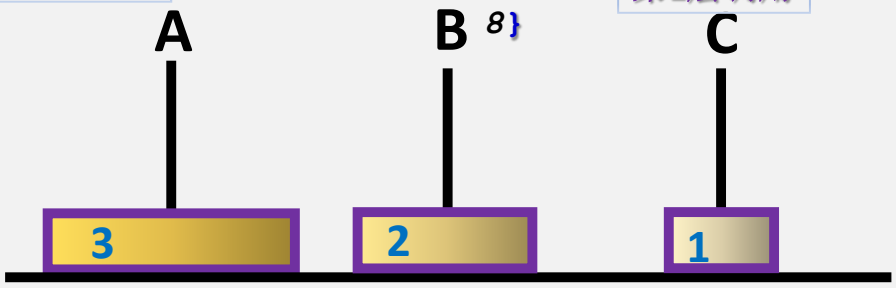
```
void hanoi(int n, ←实参 2
           char x, ←--A
           char y, ←--C
           char z  ←--B
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参:
4    hanoi(n-1,x,z,y);
     //实参:
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

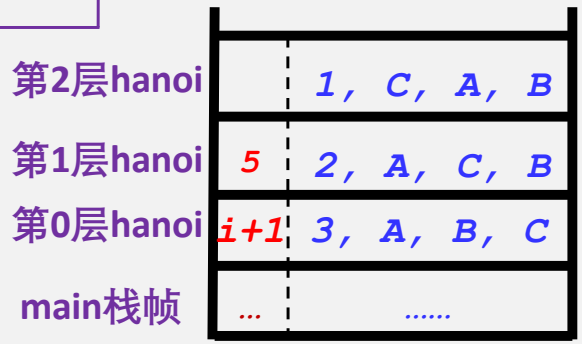
1 C A B

第1层调用

```
void hanoi(int n, ←实参
           char x, ←--
           char y, ←--
           char z  ←--
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参:
4    hanoi(n-1,x,z,y);
     //实参:
5    move(  x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第2层调用





hanoi(3, A, B, C)执行过程

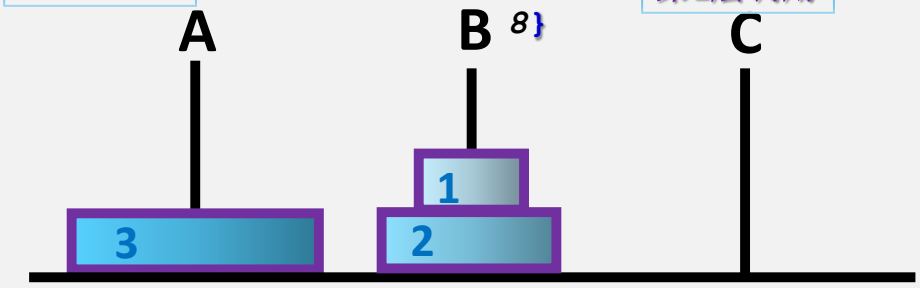
```
void hanoi(int n, ←实参: 3
           char x, ←—- A
           char y, ←—- B
           char z  ←—- C
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参: 2 A C B
4    hanoi(n-1,x,z,y);
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

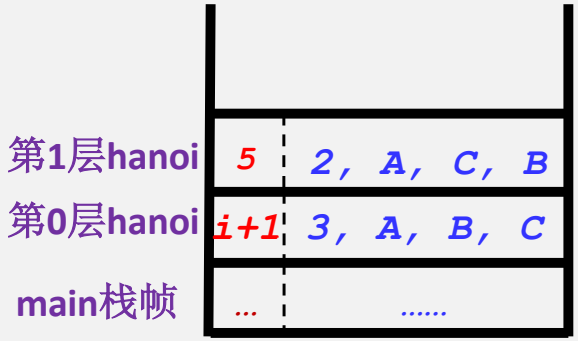
第0层调用

```
void hanoi(int n, ←实参: 2
           char x, ←—- A
           char y, ←—- C
           char z  ←—- B
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参:
4    hanoi(n-1,x,z,y);
     //实参:
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

1 C A B

第1层调用





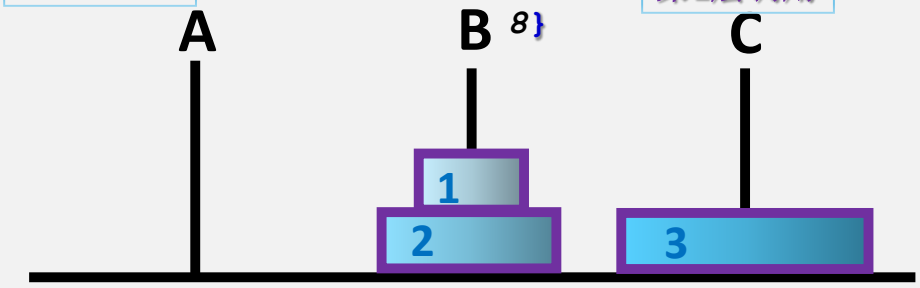
hanoi(3, A, B, C)执行过程

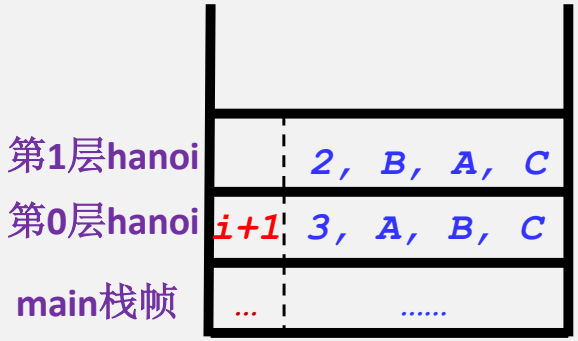
```
void hanoi(int n, ←实参: 3
           char x, ←——A
           char y, ←——B
           char z  ←——C
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参: 2 A C B
4    hanoi(n-1,x,z,y);
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第0层调用

```
void hanoi(int n, ←实参:
           char x, ←——
           char y, ←——
           char z  ←——
          ) {
1  if (1 == n)
2    move(x,1,z);
3  else {
     //实参:
4    hanoi(n-1,x,z,y);
     //实参:
5    move(x,n,z);
6    hanoi(n-1,y,x,z);
7  }
8}
```

第1层调用





3.4 队 列

3.4.1 抽象数据类型队列的定义

和栈相反，**队列**（queue）是一种**先进先出**（first in first out，缩写为FIFO）的线性表。只允许在表的一端进行插入，而在另一端删除元素。这和日常生活中的排队是一致的，最早进入队列的元素最早离开。

在队列中，允许插入的一端叫做**队尾**（rear），允许删除的一端则称为**队头**（front)。对于队列$Q=(a_1, a_2,\ldots, a_n)$，a_1就是队头元素，a_n则是队尾元素。

队列中的元素是按照$a_1, a_2,\ldots, a_n$的顺序进入的，退出队列也只能按照这个次序依次退出；也就是说，只有在$a_1, a_2,\ldots, a_{n-1}$都离开队列之后，a_n才能退出队列。

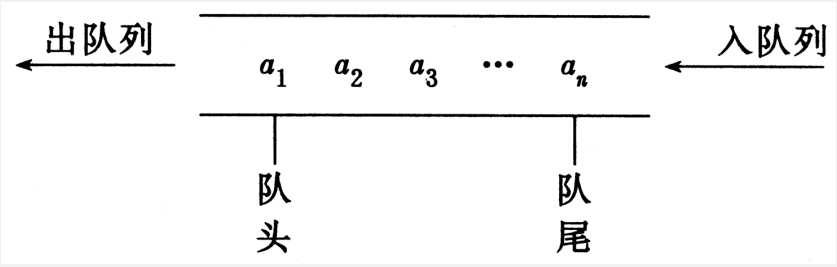


图3.8 队列的示意图

队列的抽象数据类型定义

清華大學出版社 TSINGHUA UNIVERSITY PRESS

ADT Queue {

数据对象：**D = {** a_i | $a_i \in ElemSet, i=1,2,\cdots,n,\ n\geq 0$ **}**

数据关系：**R1 = {** $<a_{i-1},a_i>$ | $a_{i-1},a_i \in D,\ i=2,\cdots,n$ **}**

约定其中a_1端为队列头，a_n端为队列尾。

基本操作：

InitQueue(Q)；

操作结果：构造一个空队列Q。

FreeQueue(Q)；

初始条件：队列Q已存在。

操作结果：队列Q被回收，不再存在，返回**NULL**。

ClearQueue(Q)；

初始条件：队列Q已存在。

操作结果：将Q清为空队列。

QueueEmpty(Q)；

初始条件：队列Q已存在。

操作结果：若Q为空队列，则返回**TRUE**，否则**FALSE**。

QueueLen(Q)；

初始条件：队列Q已存在。

操作结果：返回Q的元素个数，即队列的长度。

GetHead(Q)；

初始条件：Q为非空队列。

操作结果：返回Q的队头元素。

EnQueue(Q, e)；

初始条件：队列Q已存在。

操作结果：插入元素e为Q的新的队尾元素。

DeQueue(Q)；

初始条件：Q为非空队列。

操作结果：删除Q的队头元素，并返回其值。

QueueTrav(Q, visit())；

初始条件：Q已存在且非空。

操作结果：从队头起依次对Q每个数据元素调用函数**visit()**。一旦**visit()**失败、则操作失败。

}ADT Queue

队列的操作与栈的操作类似，也有9个，不同的是删除是在表的头部（即队头）进行。

3.4.2 链队列——队列的链式表示和实现

类似线性表，队列也有两种存储表示。链表表示的队列简称为**链队列**，如图3.10所示。

一个链队列显然需要两个分别指示**队头**和**队尾**的**指针**（分别称为**头指针**和**尾指针**）才便于在队头和队尾的操作。

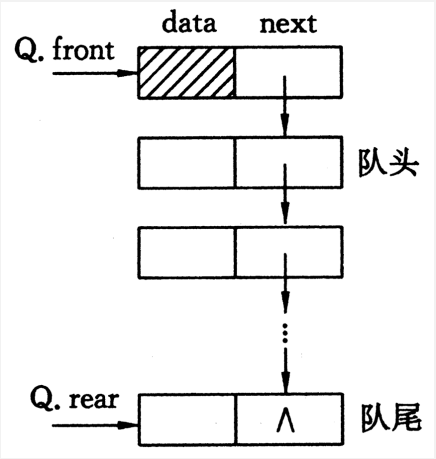


图3.10 链队列示意图

为操作方便，也给链队列添加一个**头结点**，并令头指针指向头结点。空的链队列的判决条件是“头指针和尾指针均指向头结点”，如图3.11(a)所示。

链队列的操作是单链表的插入和删除操作的特殊情况，但要同时维护头、尾两个指针，图3.11(b)~(d)展示了进行这两种操作时指针变化的情况。

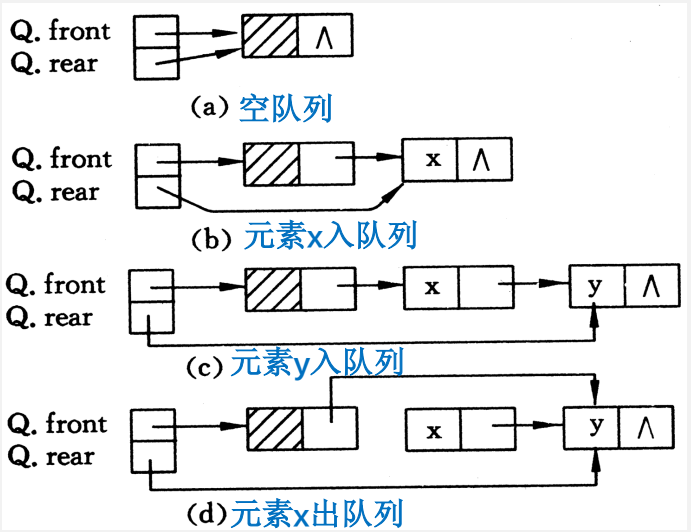


图3.11 队列运算指针变化状况

链队列出队算法

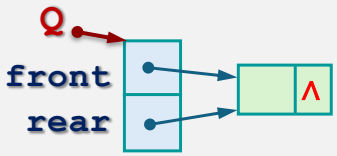
```
ElemType DeQueueL(LinkQueue Q) {   // 出队列
   if (!Q || Q->front==Q->rear) return nullE;   // 若Q空或队空，则返回nullE
   QueuePtr p = Q->front->next;                 // p指向队头元素结点
   ElemType e = p->data;                        // 队头元素值赋给e
   Q->front->next = p->next;                    // 删除队头元素结点
   if (Q->rear==p) Q->rear = Q->front;          // 若删除的也是队尾结点，则队列变空
   free(p);   return e;                         // 释放被删结点，返回退队元素值
}
```

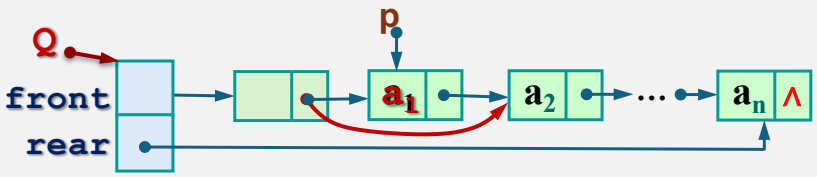
链队列出队操作要考虑三种情况。

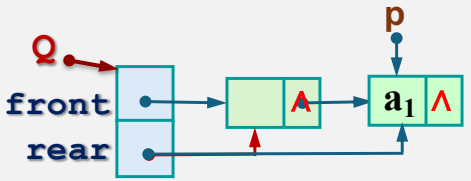
情形1： 队列长度为0
返回nullE

情形2： 队列长度大于1
（1）用指针p 指向队首的a_1元素结点。
（2）用e保存即将出队的元素值。
（3）头结点next指向出队元素后继a_2。
（4）释放出队元素p结点，返回e。

情形3： 队列长度为1
（1）执行情形2的前3步。
（2）置队尾指针也指向头结点
（3）同情形2的第4步







3.4.3 循环队列——队列的顺序表示和实现

类似顺序栈，除了用一组地址连续的存储单元从头到尾依次存放元素之外，需要两个指针front和rear分别指示队头元素及队尾元素位置。

约定：初始建空队列时，令front=rear=0，

每当插入新的队尾元素，“尾指针rear增1”；

每当删除了队头元素，“头指针front增1”。

在非空队列中，头指针始终指向队列头元素，

而尾指针始终指向队列尾元素的下一个位置。

假如队列被分配的空间容量为6，队列状态如图3.12(d)，若有新元素入队，则因数组越界而导致程序运行出错。此时不宜如顺序栈扩容，队列实际可用空间未占满。

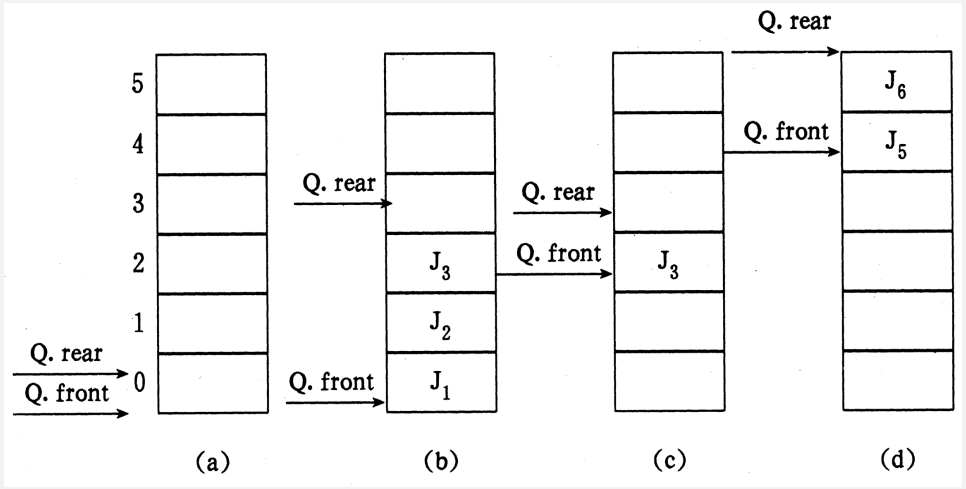


图3.12 头、尾指针和队列中元素之间的关系
(a) 空队列； (b) J1、J2和J3相继入队列； (c) J1和J2相继被删除；
(d) Ja、Js和J6相继插入队列之后J3及J被删除

清華大學出版社
TSINGHUA UNIVERSITY PRESS

较巧妙的办法是将顺序队列臆想为一个环状的空间，如图3.13所示，称之为**循环队列**。指针和元素间关系不变，如图3.14(a)所示的循环队列中，头元素是J_1，尾元素是J_5，之后J_6、J_7和J_8相继入队，则队列空间均被占满，如图3.14(b)所示，此时`Q.front==Q.rear`为真；

反之，若J_3、J_4和J_5相继从图3.14(a)的队列中删除，使队列呈“空”的状态，如图3.14(c)所示。此时也关系式`Q.front==Q.rear`也成立。

可见，只凭`Q.front==Q.rear`无法判别队列空间是“空”还是“满”。可有多种处理方法：

- 增设一个标志位以区别队列是“空”还是“满”；
- 增设一个长度域，空队列的长度为0；
- **少用一个元素空间，约定以“队列头指针在队列尾指针的下一位置（指环状的下一位置）上”作为队列为“满”的状态判据。**

前两种方法的缺点是每当有入队或出队操作，必须维护标志域或长度域的值。**第三种方法以一个元素单元的空间代价换得对队列空、满状态判别的便利。**

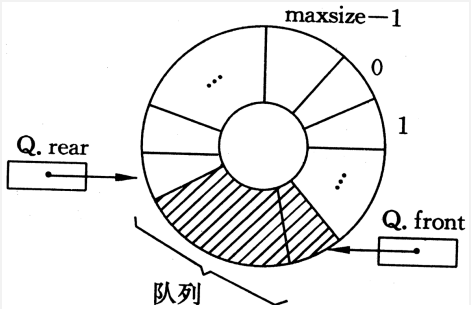


图3.13　循环队列示意图

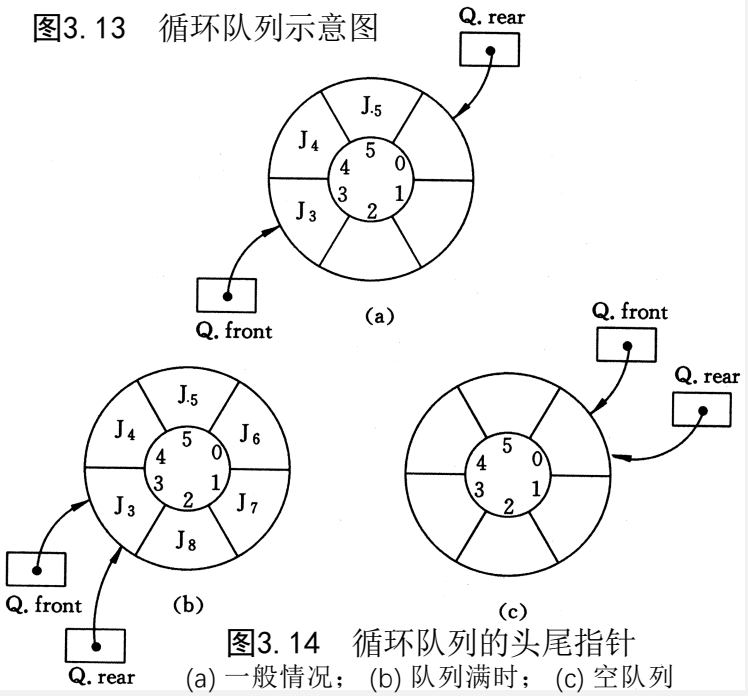


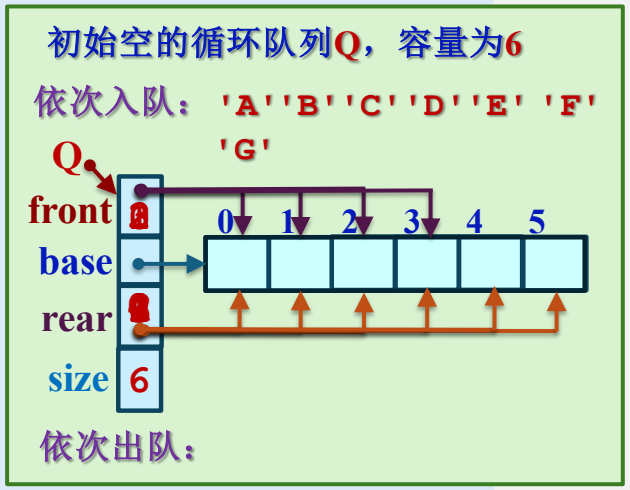
图3.14　循环队列的头尾指针
(a) 一般情况；(b) 队列满时；(c) 空队列

循环队列类型定义和主要操作的实现

```
typedef struct {
  ElemType *base;    // 初始化时动态分配元素存储空间的基址
  int       front;   // 头指针，若队列不空，指向队列头元素
  int       rear;    // 尾指针，若队列不空，指向队列尾元素的下一个位置
  int       size;    // 被分配的存储空间元素容量
} *SqQueue;          // 循环队列指针类型

Status EnQueue(SqQueue Q, ElemType e) { // 入队，插入元素e为Q的新队尾元素
  if (!Q || (Q->rear+1)%Q->size == Q->front) return ERROR;  // Q不存在或满则报错
  Q->base[Q->rear] = e;                          // 元素e入队
  Q->rear = (Q->rear + 1) % Q->size; // 尾指针（循环）增1
  return OK;
}

ElemType DeQueue(SqQueue Q) {    // 退队
  // 若队列存在且不空，则删除并返回Q的队头元素，否则返回ERROR
  if (!Q || Q->front==Q->rear) // 若队列Q不存在或空
    return nellE;                          // 则报错
  ElemType e = Q->base[Q->front];       // 退队元素值赋给e
  Q->front = (Q->front + 1) % Q->size; // 头指针(循环)增1
  return e;                                      // 返回退队元素值
}
```



3.5 离散事件模拟

建议在AnyviewC可视交互跟踪模拟过程

某银行3个柜台对外接待顾客，银行开门营业后不断有顾客进入银行。每个柜台接待一个顾客，顾客需在每个柜台前依次排队。刚进入银行的顾客，若有柜台正空闲，可上前办理业务；反之，若3个柜台均有顾客所占，一般会选择在人数最少的队伍后面**排队**。

现要编制程序模拟银行业务活动，计算一天中顾客在银行逗留的**平均时间**，以分析柜台服务的繁忙度。

为计算这个平均时间，需要掌握每个顾客**到达**银行和**离开**银行这两个**时刻**，后者减去前者即为每个顾客在银行的逗留时间；所有顾客逗留时间的总和除以当天进入银行的顾客数就是所求的平均时间。顾客到达和离开银行这两个时刻发生的事情称为“**事件**”。模拟程序将按事件发生的先后顺序进行处理，这种模拟称作**事件驱动模拟**。**算法3.6**描述了上述银行顾客和办理业务的离散事件驱动模拟程序。

```
void Bank_Simulation() {   // 算法 3.6 银行业务模拟，并统计当天顾客在银行逗留的平均时长
   OpenForDay();                                        // 开门营业前的初始化
   while (MoreEvent()) {                                // 获取事件（若无则循环结束）
      switch (EventKind()) {                            // 区分事件种类
         case 'A': CustomerArrivel();      break;       // 处理顾客到达事件
         case 'D': CustomerDeparture();    break;       // 处理顾客离开事件
      }
   }
   CloseForDay();                                       // 打烊（计算平均逗留时间）
}
```